

Intel·ligència Artificial

Informe Pràctica 1: Cerca Local

Maria Caldart Deu
Nathaniel Cruz Pascual
Aleix Molins Teulé

Taula de continguts

1. Descriptiva.....	3
1.1. Identificació del problema.....	3
1.2. Estat i representació.....	3
1.3. Operadors.....	5
1.4. Heurística.....	6
1.5. Estat inicial.....	7
2. Experiments.....	8
2.1. Experiment 1.....	8
2.1.1. Plantejament.....	8
2.1.2. Dades experimentals.....	8
2.1.3. Anàlisi.....	10
2.2. Experiment 2.....	10
2.2.1. Plantejament.....	10
2.2.2. Dades experimentals.....	11
2.2.3. Anàlisi.....	13
2.3. Experiment 3.....	13
2.3.1. Plantejament.....	13
2.3.2. Dades experimentals.....	15
2.3.3. Anàlisi.....	18
2.4. Experiment 4.....	18
2.4.1. Plantejament.....	18
2.4.2. Dades experimentals.....	20
2.4.3. Anàlisi.....	22
2.5. Experiment 5.....	23
2.5.1. Plantejament.....	23
2.5.2. Dades experimentals.....	25
2.5.3. Anàlisi.....	28
2.6. Experiment 6.....	29
2.6.1. Plantejament.....	29
2.6.2. Dades experimentals.....	31
2.6.3. Anàlisi.....	32
2.7. Experiment 7.....	33
2.7.1. Plantejament.....	33
2.7.2. Dades experimentals.....	34
2.7.3. Anàlisi.....	36
3. Conclusió final.....	36
4. Treball d'innovació.....	38

1. Descriptiva

1.1. Identificació del problema

El problema ens planteja un escenari de gestió d'emergències on, després d'una catàstrofe, cal rescatar diversos grups de persones aïllades utilitzant una flota d'helicòpters distribuïts en diferents centres de comandament. L'objectiu principal és planificar les rutes de rescat òptimes per minimitzar el temps total d'operació de la flota i, en una fase més avançada, prioritzar el rescat dels grups amb persones ferides.

Ens trobem davant d'un problema d'optimització combinatòria, molt similar al clàssic problema d'enrutament de vehicles (Vehicle Routing Problem - VRP), on hem de satisfer restriccions estrictes: capacitat màxima de l'helicòpter (15 persones) i un màxim de grups per viatge (3 grups). Donat l'elevat nombre de grups, centres i helicòpters, l'espai d'estats teòric és d'una magnitud colossal. Una cerca no informada o exhaustiva resultaria totalment inabordable a nivell computacional a causa de l'explosió combinatòria. Per aquest motiu, la resolució mitjançant algorismes de Cerca Local (com Hill Climbing o Simulated Annealing) és l'enfocament més adequat. Per maximitzar l'eficiència, el nostre model només navega per l'espai de solucions factibles (complint sempre les restriccions logístiques), permetent trobar una configuració subòptima d'alta qualitat en un temps raonable i sacrificant la completesa a canvi de viabilitat computacional.

1.2. Estat i representació

Definició d'estat

En el context d'aquest problema de Cerca Local, un estat es defineix com a una solució completa i vàlida del problema en un instant de temps donat. D'una manera més concreta, un estat conté l'assignació total dels N grups de persones a rescatar als diferents helicòpters disponibles, estructurats en viatges seqüencials en els quals s'especifica l'ordre exacte de recollida i la ruta que segueix cada vehicle de rescat, i respectant sempre les restriccions del problema, que són màxim 15 persones repartides en un màxim de 3 grups dins d'un mateix helicòpter a la vegada, és a dir, per viatge.

Estructures de dades utilitzades i la seva justificació

Per tal de representar aquest estat dins del nostre codi, hem dissenyat la classe ProbBoard, evitant la utilització d'estructures excessivament complexes que puguin alentir el procés. Hem dividit la informació en dues categories, dades estàtiques i dinàmiques, per així poder maximitzar l'eficiència tant espacial com temporal.

D'aquesta manera, l'estat es representa principalment mitjançant les següents estructures:

1. **Dades Estàtiques:** Tota la informació de l'escenari que no muta entre els diferents estats s'ha declarat com a static. Això inclou un objecte de la classe Grupos, que conté el conjunt de tots els grups (totsElsGrups), l'objecte dels centres (totsElsCentres) i el nombre total d'helicòpters (numHelicopters), que en aquest cas és un enter.
 - **Justificació:** El fet de declarar aquestes variables com a estàtiques ens permet garantir que les dades només es guarden en memòria un únic cop i que, tal i com nosaltres volem, es comparteixen entre totes les instàncies de

ProbBoard que es generin. D'aquesta manera evitem copiar les coordenades i característiques de cada grup i centre a cada estat generat, i així evitem esgotar l'espai a la memòria amb dades infinitament redundants.

- Comporta una gran eficiència espacial.
- 2. **Dades Dinàmiques:** La nostra estructura de dades principal, que és la que ens permet assignar les rutes de cada helicòpter, es representa mitjançant una estructura de llistes niuades:

Java

```
public ArrayList<ArrayList<ArrayList<Integer>>> assignacioHelicopters;
```

Aquesta estructura té diferents nivells:

- Nivell 1: Cada helicòpter de la flota.
- Nivell 2: Els diferents viatges (anada i tornada a la base) que fa un determinat helicòpter.
- Nivell 3: Els identificadors dels grups rescatats en aquell viatge concret.
 - **Justificació:** Com que guardem només els identificadors de cada grup en comptes de l'objecte sencer, la clonació de l'estat per generar els successors es converteix en una funció molt ràpida i lleugera, ja que treballa amb enters i no amb estructures més complexes (eficiència espacial). A més, el fet de tenir tots els viatges dels helicòpters de manera seqüencial permet que el càlcul de la funció heurística sigui molt més eficient i ràpid, ja que només hem de recórrer-les en ordre per calcular les distàncies de vol (eficiència temporal).

Mida de l'espai de cerca

En aquest subapartat analitzarem la complexitat matemàtica d'aquest espai dividint la creació d'un estat en dues dimensions:

- **Assignació de grups:** Si tenim G grups a rescatar i H helicòpters, el simple fet d'assignar cada grup a un vehicle, sense tenir en compte en quin viatge o ordre, genera un espai de H^G combinacions possibles. En el nostre escenari base de les proves (100 grups i 5 helicòpters), això suposa 5^{100} combinacions bàsiques.
- **Ordenació de les rutes:** Un cop assignats els grups a un helicòpter, l'ordre en què el pilot els va a rescatar altera radicalment la distància de vol i, per tant, el cost heurístic. Si un helicòpter té assignats n grups, existeixen $n!$ rutes de rescat diferents per a aquell mateix conjunt de persones.

La mida total de l'espai de cerca és el resultat del producte entre les assignacions (H^G) amb totes les seves permutacions de ruta ($n!$). Aquest creixement factorial i exponencial provoca una severa explosió combinatòria. Aquesta demostració matemàtica evidencia que el nombre d'estats és inabastable i justifica plenament la necessitat d'utilitzar una representació eficient de l'estat, així com l'obligatorietat d'emprar algorismes de Cerca Local probabilístics per sobre de cerques exhaustives.

$$\text{Espai de cerca} = H^G \cdot n!$$

Nota sobre la cota superior de l'espai de cerca

Com a petit matís teòric, és convenient destacar que si avaluéssim l'espai de cerca absolut en el seu pitjor cas (per exemple, si l'algorisme fos tan ineficient d'assignar la totalitat dels G grups a un únic helicòpter), les permutacions de ruta per a aquest vehicle s'elevarien a $G!$ ($100!$ en el nostre escenari base). Aquesta cota superior demostra la inabastabilitat del problema i el risc real de col·lapse computacional si no s'apliquen restriccions severes durant la generació d'estats.

1.3. Operadors

Per poder recórrer l'espai d'estats i permetre que els algorismes de cerca local evolucionin des d'una solució inicial fins a una de millor, hem dissenyat dos operadors que defineixen el veïnatge d'un estat. El nostre procés de raonament va ser pensar en els moviments mínims necessaris per reestructurar la càrrega i les rutes respectant de forma estricta les restriccions del problema.

1. **Moure Grup:** Aquest operador extreu un grup de persones d'un viatge concret d'un helicòpter origen i l'assigna a un helicòpter destí. Si el darrer viatge de l'helicòpter destí té prou capacitat s'hi afegeix; si no, es crea un viatge nou.

Raonament: Aquest operador actua com a mecanisme de balanceig de càrregues. Permet descongestionar helicòpters amb rutes molt llargues i pot arribar a buidar completament un helicòpter si resulta més eficient deixar-lo a terra. Altera la distribució del nombre de viatges.

2. **Intercanviar Grups:** Aquest operador agafa un grup assignat a un helicòpter A i un grup d'un helicòpter B, i els intercanvia, sempre que l'estat resultant sigui vàlid (≤ 15 persones).

Raonament: Està pensat per optimitzar distàncies geogràfiques pures sense alterar l'estructura de viatges de l'estat. Permet "polir" les rutes apropant grups als centres corresponents.

Observacions rellevants:

Durant el disseny ens vam plantejar si un sol operador generaria un veïnatge prou ric. A l'Experiment 1 vam comprovar que l'ús aïllat generava un espai d'estats poc connectat, deixant l'algorisme atrapat en òptims locals deficientes. Combinar Tots Dos operadors augmenta notablement el factor de ramificació (generant un veïnatge molt més ampli), però dota l'algorisme de la connectivitat necessària per escapar de valls poc prometedores i assolir solucions d'alta qualitat.

Anàlisi del factor de ramificació:

Per justificar l'impacte temporal d'aquests operadors, hem analitzat el seu factor de ramificació (nombre de successors que generen per cada estat). Donat un escenari amb G grups a rescatar i H helicòpters en total:

- **Moure Grup:** En el pitjor dels casos, podem agafar qualsevol dels G grups i intentar moure'l a qualsevol dels H helicòpters (excloent el seu). Això genera un factor de ramificació aproximat de $O(G \cdot H)$.
- **Intercanviar Grups:** Aquest operador intenta intercanviar totes les parelles de grups possibles. Matemàticament, això són combinacions de G elements agafats de 2 en 2, és a dir, $(G \cdot (G - 1)) / 2$. Això genera un factor de ramificació de $O(G^2)$.

En utilitzar Tots Dos operadors combinats (com vam decidir fixar a l'Experiment 1), el factor de ramificació total de la nostra funció successora és aproximadament $O(G^2 + G \cdot H)$. Com que el nombre de grups G (ex: 100) sol ser molt més gran que el d'helicòpters H (ex: 5), l'operador que domina i fa créixer l'esforç computacional de cada iteració és l'intercanvi ($O(G^2)$). Tanmateix, aquest dens veïnatge és justament el que ens permet esquivar òptims locals i assolir solucions d'alta qualitat.

1.4. Heurística

Anàlisi dels factors del problema

Per tal d'avaluar la qualitat d'un estat, hem de tenir en compte els diferents factors físics i temporals que influeixen en l'operació de rescat, segons l'enunciat:

1. **Temps de vol:** Derivat de la distància euclidiana entre coordenades (grups i centres) assumint una velocitat constant de 100 km/h.
2. **Temps de càrrega (Prioritat):** Depèn del tipus de passatger. Els grups de Prioritat 1 (amb ferits) triguen 2 minuts per persona a pujar a l'helicòpter, mentre que els de Prioritat 2 (sans) triguen 1 minut per persona.
3. **Penalitzacions i descàrregues:** Cada viatge addicional que ha de fer un mateix helicòpter (és a dir, a partir del seu segon vol) suma una penalització fixa de 10 minuts per a la descàrrega de persones i preparació a la base.

Descripció i justificació de les heurístiques implementades

A partir d'aquests factors, dins la classe ProbBoard hem dissenyat les funcions heurístiques que l'algorisme haurà de minimitzar:

- **Heurística 1 (Temps total de la flota):** El mètode heuristic() calcula la suma estricta de tots els minuts emprats per tots els helicòpters en completar les seves rutes respectives. Aquesta funció és la representació matemàtica directa del primer criteri de l'enunciat.
- **Heurística 2 (Temps del pitjor rescat de Prioritat 1):** El mètode heuristic2() calcula exclusivament el temps que transcorre des de l'inici del dia fins que l'últim grup amb ferits (Prioritat 1) arriba a la base sa i estalvi. Mesura el "pitjor cas" per a les persones crítiques.
- **Heurística Combinada. Ponderació:** Per complir amb el segon criteri de l'enunciat (minimitzar el temps total però minimitzant també el temps de rescat dels ferits), hem creat el mètode ponderacioHeuristics().

Java

```
public double ponderacioHeuristics() {  
    return heuristic() + 64*heuristic2();  
}
```

Justificació de les ponderacions i efectes en la cerca

Durant el desenvolupament, vam contemplar diverses possibilitats que ens van fer prendre la decisió d'utilitzar una suma ponderada al Criteri 2 (multiplicant per 64 el temps dels ferits):

- Si sumàvem els dos valors sense ponderar (sumaHeuristics() a ProbBoard), l'algorisme de cerca donava massa pes al temps global. Preferia deixar un grup de ferits esperant hores si això li estalviava uns pocs minuts de vol a un altre helicòpter.
- D'altra banda, hem vist que la minimització màxima del temps de rescat dels ferits (el punt en què encara que donem més pes a heuristic2() el temps no baixa) és quan multipliquem heuristic2() per 64. Per tant, el temps de rescat de ferits és minimitzat amb aquest valor, i és la ponderació que millor compleix el criteri 2 (minimitzar el temps de rescat total i el temps de rescat dels ferits).

Anàlisi del cost operacional

Hem de tenir en compte l'eficiència en el càlcul de l'heurística, ja que l'algorisme l'executa milers de vegades. Gràcies a la nostra representació de l'estat en llistes ordenades de viatges, l'algorisme només necessita recórrer la llista de grups rescatats de forma seqüencial, sumant distàncies pas a pas, tal com hem descrit a l'apartat 1.2 quan parlàvem de les dades dinàmiques. Com que cada grup a rescatar es visita exactament una vegada durant el càlcul, el cost operacional temporal és lineal respecte al nombre de grups, on G és el nombre total de grups assignats.

$$Cost = O(G)$$

1.5. Estat inicial

L'estat inicial constitueix el punt de partida dins l'espai d'estats des d'on l'algorisme de cerca local comença la seva exploració. En problemes d'optimització combinatoria complexos com aquest, la qualitat d'aquesta solució llavor és un factor crític que condicionarà fortament la velocitat de convergència cap a un òptim local.

Amb l'objectiu de dotar l'algorisme del millor punt de partida possible, hem dissenyat dues estratègies de generació. Ambdues garanteixen que l'estat generat sigui sempre factible (respectant estrictament les restriccions d'un màxim de 15 persones i 3 grups per viatge):

1. Generació Aleatòria (*Baseline*): Aquesta estratègia recorre la llista de grups a rescatar i assigna cadascun d'ells a un helicòpter escollit de forma totalment atzarosa.

- Procés de raonament: L'objectiu d'aquesta inicialització era servir com a línia base de comparació. És una inicialització no informada que genera rutes de rescat caòtiques (un helicòpter pot haver de creuar tot el mapa diverses vegades en un sol viatge). La vam pensar per posar a prova la capacitat "en brut" de l'algorisme per redreçar solucions pèssimes.
2. Generació Avariciosa (*Greedy*): Aquesta estratègia introdueix coneixement específic del domini. Per a cada grup a rescatar, es calcula la distància cap a tots els centres de comandament i s'assigna al primer helicòpter disponible del centre més proper.
- Procés de raonament: Sabent que la distància de vol és el factor que més encareix el temps de rescat, vam pensar que agrupar geogràficament els rescats des del minut zero generaria rutes intuïtivament eficients, alliberant l'algorisme d'haver de fer ajustos geogràfics obvis.

2. Experiments

2.1. Experiment 1

2.1.1. Plantejament

En aquest primer experiment volem comprovar quin conjunt d'operadors (o la combinació d'aquests) és el més adequat per minimitzar el temps total de rescat de la flota (Funció Heurística 1).

La metodologia consisteix a utilitzar un escenari base de 5 centres, 1 helicòpter per centre i 100 grups a rescatar, utilitzant l'algorisme de Hill Climbing i partint d'una solució inicial completament aleatòria. S'executaran 10 proves amb diferents llavors fixes per a tres configuracions: només utilitzar l'operador "Moure", només "Intercanviar", i utilitzar "Tots Dos" alhora.

La nostra hipòtesi inicial és que la configuració amb Tots Dos operadors aconseguirà trobar les solucions amb un cost més baix, ja que disposa d'un factor de ramificació més gran i l'espai d'estats està més connectat per evitar òptims locals pobres. No obstant això, esperem que aquesta configuració sigui la que requereixi més temps d'execució.

2.1.2. Dades experimentals

Configuració d'Operadors	Cost Mitjà	Final	Temps (ms)	Mitjà	Passos (Nodes)	Mitjans

Tots Dos Operadors	2.737,80	1.761,9	94,6
Només Moure	3.137,66	74,3	58,1
Només Intercanviar	2.886,98	1.343,5	87,3

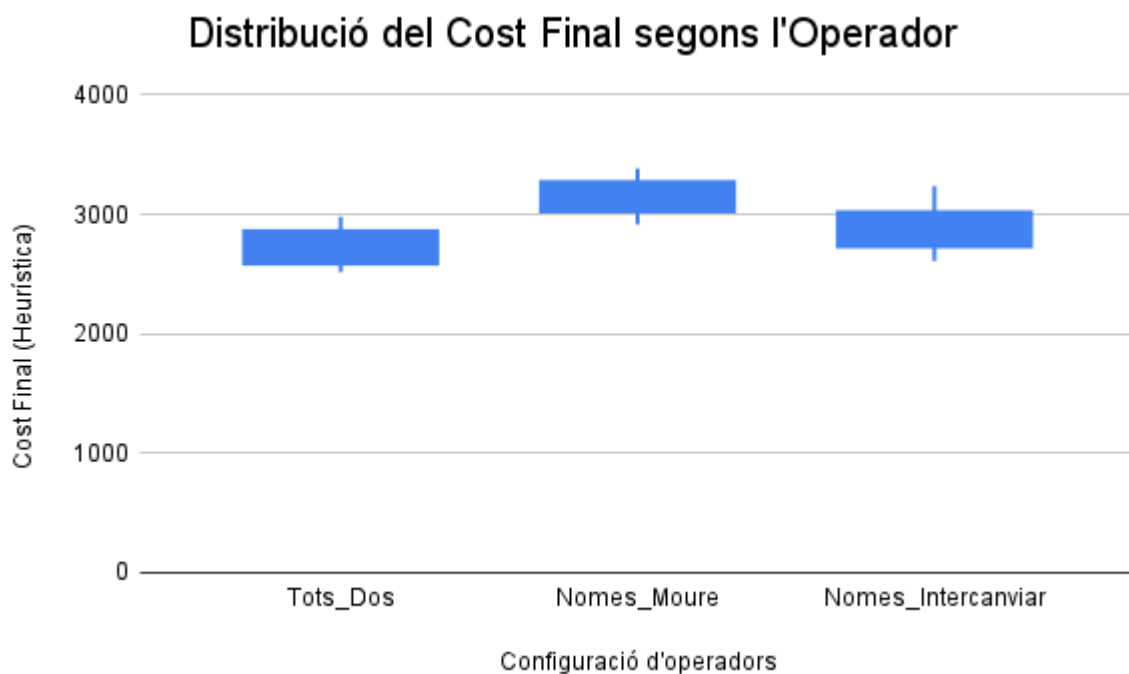


Figura 1: Diagrama d'espelmes mostrant la distribució del Cost Final assolit per cada configuració d'operadors (10 execucions per mostra)

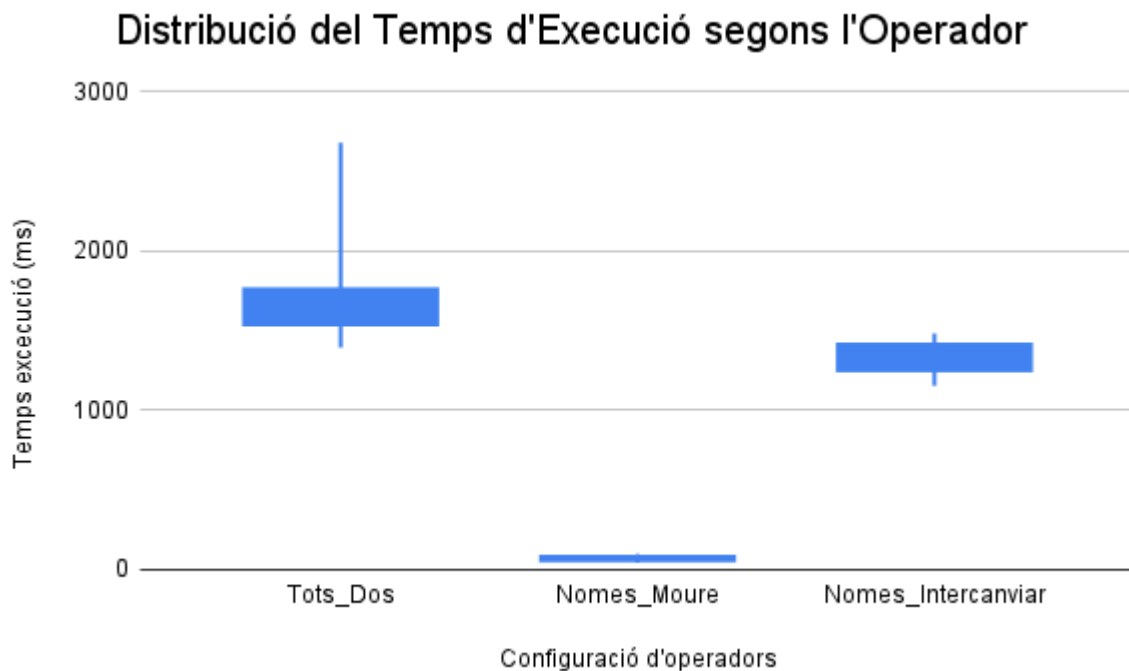


Figura 2: Diagrama d'espelmes mostrant la distribució del Temps d'execució (ms) segons l'operador utilitzat.

2.1.3. Anàlisi

Observant els resultats i els gràfics de distribució, la nostra hipòtesi es confirma totalment. Tal com s'evidencia a la Figura 1, pel que fa a la bondat de la solució, l'ús combinat de Tots Dos operadors assoleix el cost mitjà més baix (2.737,80) i presenta una gran estabilitat entre les diferents llavors, superant clarament l'operador de Només Moure (3.137,66), el qual es queda atrapat ràpidament en òptims locals. Teòricament, això s'explica perquè l'ús d'un sol operador no garanteix que l'espai d'estats estigui fortament connectat, impedit arribar a les millors solucions.

D'altra banda, la Figura 2 il·lustra perfectament l'impacte directe del factor de ramificació sobre el cost temporal: Només Moure genera menys successors i triga amb prou feines 74 ms, mentre que Tots Dos operadors necessita al voltant de 1.761 ms per avaluar el dens veïnatge de cada iteració. Tot i aquesta diferència geomètrica en el temps, 1,7 segons continua sent un cost computacional extremadament baix i assumible per a un algorisme de Hill Climbing. Per tant, prioritzant la minimització del temps de rescat i la qualitat de la solució, decidim fixar l'ús de Tots Dos operadors per a la resta de la pràctica.

2.2. Experiment 2

2.2.1. Plantejament

L'objectiu d'aquest segon experiment és determinar si partir d'una solució inicial generada de forma intel·ligent millora el rendiment i el resultat final de la cerca en comparació a una inicialització aleatòria. La metodologia consisteix a avaluar el mateix escenari base (5 centres, 1 heli, 100 grups) usant l'algorisme de Hill Climbing i, segons les conclusions de l'experiment anterior, fixant l'ús de Tots Dos operadors. Compararem l'estratègia Aleatòria amb l'estratègia Avariciosa (que assigna els grups a l'helicòpter del centre més proper) mitjançant 10 repeticions amb les mateixes llavors. La nostra hipòtesi és que l'estratègia Avariciosa generarà un estat inicial amb un cost molt inferior, fet que permetrà al Hill Climbing començar la cerca en una zona molt més prometedora de l'espai d'estats. En conseqüència, esperem obtenir una millor solució final i, alhora, reduir significativament el nombre d'iteracions i el temps de còmput.

2.2.2. Dades experimentals

Estratègia Inicial	Cost Inicial Mitjà	Cost Final Mitjà	Temps Mitjà (ms)	Passos Mitjans (Nodes)
Aleatòria	4.520,64	2.744,77	1.668,0	95,2
Avariciosa	3.026,00	2.649,89	820,6	48,0

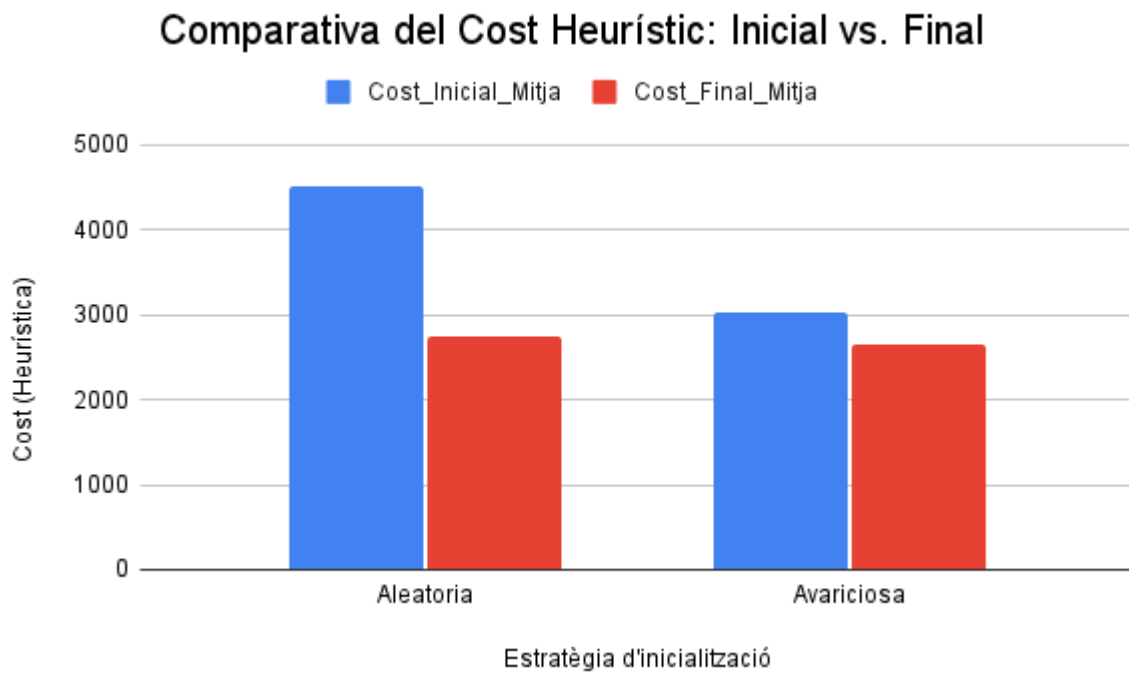


Figura 3: Comparativa de les mitjanes del Cost Inicial respecte al Cost Final assolit segons l'estratègia de generació.

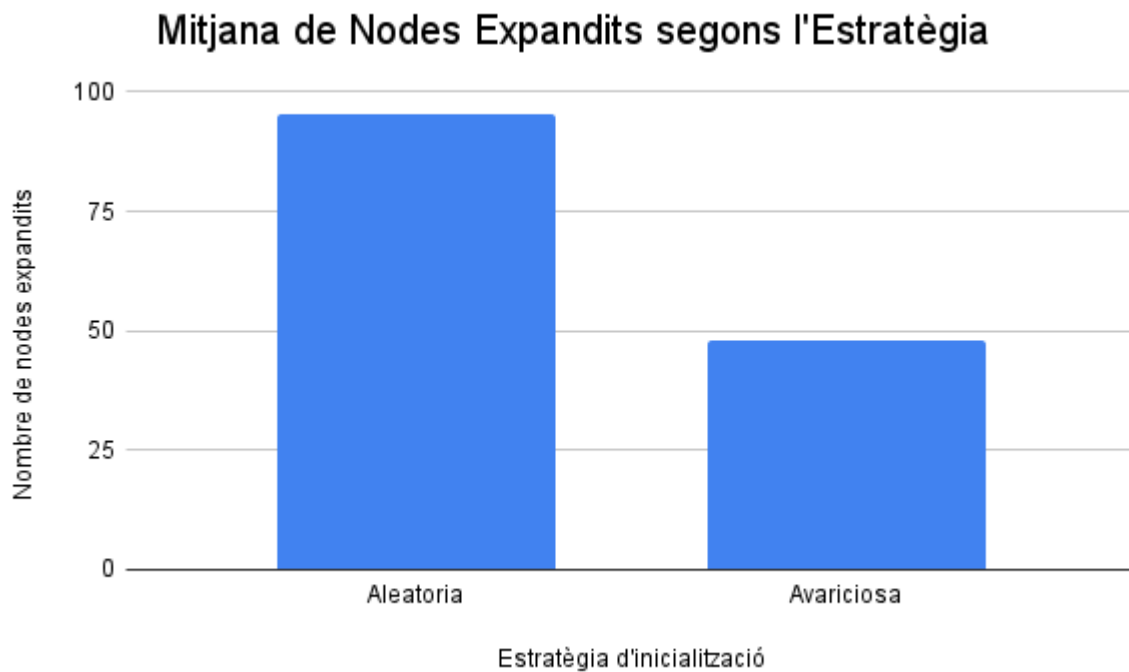


Figura 4: Mitjana de nodes expandits per l'algorisme de Hill Climbing fins a convergir en una solució.

2.2.3. Anàlisi

L'estratègia Avariciosa demostra una superioritat indiscutible en tots els paràmetres, validant la nostra hipòtesi inicial. Com es pot apreciar de forma molt visual a la Figura 3, la bondat de l'estat inicial és dràsticament millor amb l'Avariciosa (3.026,00 davant dels 4.520,64 de l'Aleatòria). Com que l'Avariciosa assigna des del principi cada grup al centre més proper, la component de distància geogràfica de l'heurística ja neix altament optimitzada.

Partir des d'un punt de l'espai de cerca tan proper a un òptim local d'alta qualitat permet que l'algorisme acabi convergint en solucions finals més eficients (2.649,89 de cost final, millorant la mitjana de l'Aleatòria). Més enllà de la qualitat, la Figura 4 demostra empíricament el benefici més destacable: l'estalvi computacional massiu. En començar la cerca pràcticament a la base d'una bona 'vall' local, l'algorisme necessita expandir exactament la meitat de nodes (48 davant 95) per convergir. Això redueix el temps mitjà a la meitat (de 1.668 ms a 820 ms). Aquest estalvi temporal serà crític de cara a escalar el problema als pròxims experiments. En conseqüència, fixem l'estratègia d'inicialització Avariciosa per defecte.

2.3. Experiment 3

2.3.1. Plantejament

L'objectiu d'aquest experiment és, a partir d'estudiar com els paràmetres de refredament propis de l'algorisme de Simulated Annealing (SA) afecten tant la qualitat de la solució final (cost heurístic) com el temps d'execució, trobar la seva configuració òptima per al nostre problema. Els paràmetres a analitzar són la "temperatura" inicial o energia (k) i el factor de descens de temperatura (λ). Un SA mal configurat perd tota la seva utilitat: si es refreda massa ràpid actua com un Hill Climbing pur, fent que es quedi atrapat en un mínim local no gaire òptim quan pot haver-hi solucions molt millors a l'altra banda del pendent; d'altra banda, si es refreda massa lentament es comporta com una cerca aleatòria caòtica.

La metodologia que seguirem consistirà a fixar les condicions del problema i variar exclusivament els paràmetres k i λ . Per tal d'avaluar exclusivament la influència d'aquests paràmetres, hem fixat la resta de variables: un escenari base de 5 centres i 100 grups, generació d'estat inicial mitjançant l'estratègia Avariciosa i 100.000 iteracions màximes per execució amb un step de refredament de 100. Per tal d'evitar el biaix de la sort en els resultats, tal com dicta l'enunciat i seguint la mateixa línia que la resta d'experiments, cada combinació de paràmetres s'avaluarà executant l'algorisme sobre 10 llavors aleatòries diferents per, posteriorment, extreure'n la mitjana tant del cost heurístic com del temps d'execució.

L'exploració es farà en tres fases:

1. **Fase de gra gruixut:** Explorarem l'espai de paràmetres amb salts d'ordre de magnitud (ex: $k \in \{1, 5, 25, 125\}$ i $\lambda \in \{0.1, 0.01, 0.001, 0.0001\}$ com hi ha a l'exemple de l'apartat 6.6.1 de l'enunciat) per acotar la regió on l'algorisme comença a convergir cap a bones solucions.
2. **Fase de gra fi:** Un cop detectada la zona prometedora, realitzarem una cerca amb valors intermedis per trobar el punt d'equilibri òptim entre el nivell d'exploració i la velocitat de convergència.

3. **Fase d'ajustament de precisió:** Finalment, per tal de garantir la màxima optimització de l'algorisme, realitzarem modificacions minúscules al voltant de la millor configuració obtinguda a la fase anterior. Això ens permetrà assegurar que els paràmetres finals triats exploten al màxim les capacitats del Simulated Annealing per al nostre escenari concret.

Donat que estem modificant el comportament intern de l'algorisme, hem plantejat dues hipòtesis diferenciades sobre com esperem que evolucionin les nostres mètriques d'estudi:

1. Hipòtesi sobre la qualitat de la solució (Cost Heurístic)

Recordem de la teoria que el Simulated Annealing accepta estats pitjors basant-se en la següent probabilitat:

$$P(\text{estat}) = e^{\left(\frac{\Delta E}{F(T)}\right)}, F(T) = k \cdot e^{-\lambda \cdot T}$$

On la temperatura T depèn directament de la nostra k inicial, i va disminuint a cada pas multiplicant-se per la constant de refredament λ . A partir d'aquesta estructura, deduïm que un valor de k excessivament baix farà que la probabilitat d'acceptar moviments dolents sigui pràcticament nul·la des del principi, estancant-se gairebé a l'instant en el primer mínim local. Per la banda contrària, si donem un valor de k extremadament alt (per exemple, $k=125$), l'algorisme tindrà una llibertat d'exploració gairebé absoluta als primers compassos.

Hipòtesi final: S'espera que com més gran sigui el valor del paràmetre k (sempre que vagi acompanyat d'un refredament lent), millors resultats absoluts s'obtindran. D'aquesta manera, gràcies a l'excés d'energia inicial s'impedirà completament que l'algorisme quedi atrapat en cap pou local de l'espai de cerca.

2. Hipòtesi sobre l'evolució del temps d'execució

A diferència de la cerca exhaustiva del Hill Climbing, el Simulated Annealing té el seu temps d'execució fortament lligat al nombre màxim d'iteracions que li permetem fer abans d'aturar-se. Donat que l'operador del SA (generar un sol veí aleatori) té un cost temporal constant d' $O(1)$, el temps d'execució vindrà donat per:

$$\text{Temps d'execució} \approx \text{Nombre de iteracions} \cdot \text{Cost heurística per iteració}$$

Com que en aquest experiment mantindrem el límit de passos completament fix per a totes les proves (100.000 iteracions màximes), el canvi de k o λ només afectarà el "camí" de solucions que pren l'algorisme, però no el nombre d'operacions matemàtiques que arriba a calcular com a màxim.

Hipòtesi final: S'espera que el temps d'execució es mantingui estable i gairebé constant en totes les proves, independentment dels valors de k i λ escollits i depenent únicament del límit d'iteracions fixat i del cost de l'heurística.

2.3.2. Dades experimentals

Per arribar als millors paràmetres, en comptes de fer una exploració exhaustiva de tots els valors possibles, ja que el cost computacional seria inassumible, hem aplicat una estratègia

de Grid Search amb refinament successiu, analitzant els resultats de cada execució per acotar la següent. Aquest mètode ens permet avaluar l'espai de solucions de forma esglaonada: comencem amb una malla de valors molt dispersa per trobar la regió prometedora i, un cop localitzada, fem 'zoom' i explorem amb més precisió només aquella zona. Totes les proves s'han fet amb un límit de 100.000 iteracions. A continuació detallem els resultats clau de cada fase:

Execució 1: Fase de gra gruixut

Vam partir provant quatre configuracions de temperatura inicial molt dispars ($k \in \{1, 5, 25, 125\}$) creuades amb diferents ordres de magnitud per al refredament ($\lambda \in \{0.1, 0.01, 0.001, 0.0001\}$). L'objectiu era veure cap a on tendia l'algorisme.

Execució 1								
K	1	1	1	1	5	5	5	5
λ	0,1	0,01	0,001	0,0001	0,1	0,01	0,001	0,0001
Temps d'execució en milisegons (mitjana)	37	265	346	351	26	259	344	342
Nodes expandits (mitjana)	7501	74601	100001	100001	7501	74601	100001	100001
Cost total (mitjana)	2709,49	2620,69	2614,94	2617,21	2715,49	2634,34	2624	2580,91
K	25	25	25	25	125	125	125	125
λ	0,1	0,01	0,001	0,0001	0,1	0,01	0,001	0,0001
Temps d'execució en milisegons (mitjana)	26	280	385	419	36	281	380	437
Nodes expandits (mitjana)	7501	74601	100001	100001	7501	74601	100001	100001
Cost total (mitjana)	2727,13	2643,57	2665,08	2602,22	2730,03	2655,12	2673,51	2604,9

D'aquesta primera ronda vam observar dues coses clau. Primer, amb valors de $\lambda=0.1$, la temperatura baixa tan ràpid que l'algorisme es congela abans d'hora, la qual cosa fa que expandi només 7.501 nodes en lloc dels 100.001 permesos, i costos pèssims per sobre de 2700. Segon, la millor configuració amb diferència va ser $k=5$ amb el refredament més lent ($\lambda=0.0001$), assolint un cost mitjà de 2580,91.

Execució 2: Fase de gra fi

Sabent que l'òptim es trobava al voltant de $k=5$, vam provar valors de $k \in \{3, 5, 7, 10\}$ i vam baixar encara més el factor de refredament fins a la meitat ($\lambda=0.00005$) per donar més temps d'exploració.

Per assegurar la fiabilitat de les dades i evitar que una llavor afortunada ens enganyés, vam repetir aquesta execució dues vegades:

- A la primera ronda (Execució 2.1), el millor valor va ser $k=5$ amb $\lambda=0.00005$ (Cost: 2558,71 min).
- A la segona ronda (Execució 2.2), el millor valor va ser $k=3$ amb $\lambda=0.00005$ (Cost: 2559,68 min).

Execució 2.1						
K	3	3	3	5	5	5
λ	0,0005	0,0001	0,00005	0,0005	0,0001	0,00005
Temps d'execució en milisegons (mitjana)	358	345	356	361	359	366
Nodes expandits (mitjana)	100001	100001	100001	100001	100001	100001
Cost total (mitjana)	2610,08	2584,09	2562,05	2606,06	2586,69	2558,71
K	7	7	7	10	10	10
λ	0,0005	0,0001	0,00005	0,0005	0,0001	0,00005
Temps d'execució en milisegons (mitjana)	362	368	359	370	388	378
Nodes expandits (mitjana)	100001	100001	100001	100001	100001	100001
Cost total (mitjana)	2611,86	2589,35	2564,6	2638,42	2589,82	2574,37
Execució 2.2						
K	3	3	3	5	5	5
λ	0,0005	0,0001	0,00005	0,0005	0,0001	0,00005
Temps d'execució en milisegons (mitjana)	335	340	334	339	332	340
Nodes expandits (mitjana)	100001	100001	100001	100001	100001	100001
Cost total (mitjana)	2615,03	2579,68	2559,68	2622,33	2576,56	2565,27
K	7	7	7	10	10	10
λ	0,0005	0,0001	0,00005	0,0005	0,0001	0,00005
Temps d'execució en milisegons (mitjana)	338	335	343	338	341	346
Nodes expandits (mitjana)	100001	100001	100001	100001	100001	100001
Cost total (mitjana)	2625,14	2585,57	2562,63	2615,25	2591,7	2562,19

Execució 3: Ajustament de precisió

Com que vam veure $k=3$ i $k=5$ com a guanyadors absoluts en les execucions anteriors, vam deduir que el punt d'equilibri es trobava entre aquests dos valors. Vam llançar una darrera execució provant estrictament $k \in \{3,4,5\}$ i reduint el descens de temperatura un pas més, fins a $\lambda=0.00002$.

Execució 3									
K	3	3	3	4	4	4	5	5	5
λ	0,0001	0,00005	0,00002	0,0001	0,00005	0,00002	0,0001	0,00005	0,00002
Temps d'execució en milisegons (mitjana)	356	343	341	336	340	351	345	342	348
Nodes expandits (mitjana)	100001	100001	100001	100001	100001	100001	100001	100001	100001
Cost total (mitjana)	2579,73	2569,45	2545,52	2593,47	2564,17	2542,41	2587,27	2568,63	2546,35

Tal com mostren les dades d'aquest ajustament final, el paràmetre $k=4$ acompanyat d'un refredament de $\lambda=0.00002$ assolix el mínim absolut amb 2542,41 minuts de cost mitjà, establint-se com la configuració guanyadora. Cal destacar que el temps d'execució es manté estable vora els 340-350 mil·lisegons independentment de la configuració un cop assolix els 100.001 nodes.

Per justificar l'elecció del $k=4$ més enllà de la simple mitjana aritmètica, hem generat uns boxplots amb les 10 llavors individuals per a la configuració guanyadora ($\lambda=0.00002$).

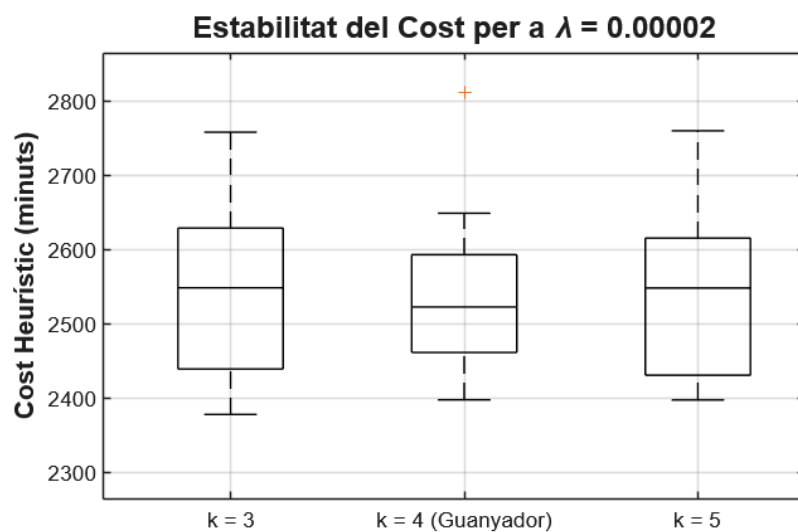


Figura 5: Estabilitat del cost heurístic per a $\lambda=0.00002$

Com podem observar al gràfic, tot i que el $k=4$ presenta un outlier on una llavor desafortunada ha generat un cost elevat, la seva "caixa" principal es troba més comprimida i situada a un nivell inferior que les de $k=3$ i $k=5$. Això demostra que la configuració **$k=4, \lambda=0.00002$** no només és més eficient en mitjana, sinó que és l'opció més robusta davant la naturalesa estocàstica del Simulated Annealing.

2.3.3. Anàlisi

Analitzant els resultats obtinguts a les dades experimentals, podem extreure diverses conclusions clares que refuten la nostra primera hipòtesi i confirmen la segona:

Tal com es pot observar a les dades, la nostra hipòtesi inicial sobre la qualitat de la solució era errònia. Els valors de k molt elevats (com $k=125$) no han donat els millors resultats, sinó que han produït costos heurístics deficients. Això es deu al fet que una temperatura inicial tan alta torna l'algorisme massa "caòtic" als primers compassos. Com que s'accepten pràcticament qualsevol moviment, fins i tot aquells que empitjoren dràsticament l'estat, l'algorisme destrueix completament les bones rutes que ens proporcionava la nostra solució inicial Avariciosa. Quan la temperatura finalment baixa i l'algorisme comença a convergir de debò, es troba en un estat tan desordenat que ja és incapaç de recuperar el temps perdut i trobar un mínim global dins del límit d'iteracions.

Al contrari, hem demostrat empíricament que l'equilibri perfecte per a aquest escenari es troba amb una temperatura inicial moderada-baixa (com ara $k=4$) acompanyada d'un refredament extremadament lent ($\lambda=0.00002$). Aquest valor de k dona la petita "empenta" energètica justa i necessària per treure l'algorisme dels mínims locals inicials en què pugui caure la inicialització Avariciosa, però sense arribar a descontrolar l'assignació. Al mateix temps, una λ de l'ordre de 10^{-5} permet que aquesta temperatura baixi molt progressivament durant les 100.000 iteracions, la qual cosa afavoreix una convergència suau, precisa i sostinguda cap a les millors solucions.

Per una altra banda, la lectura dels temps d'execució valida completament la nostra segona hipòtesi. L'ajust d'aquests paràmetres ha demostrat tenir un impacte dràstic en la "bondat" de la solució reduint el cost heurístic significativament quan s'encerta la configuració, però no ha alterat el cost temporal de manera el suficientment apreciable entre les diferents proves. El temps s'ha mantingut estable, confirmant que la complexitat temporal del Simulated Annealing en aquest domini depèn en la seva totalitat del límit fix d'iteracions i del cost constant del seu generador de successors, i no de les variacions de la temperatura.

2.4. Experiment 4

2.4.1. Plantejament

L'objectiu d'aquest quart experiment és avaluar i comparar la robustesa i l'escalabilitat dels dos algorismes de Cerca Local (Hill Climbing i Simulated Annealing) quan ens enfrontem a problemes de dimensions cada cop més grans. Ens interessa comprovar de forma empírica i analítica quin dels dos és realment viable a nivell computacional per a un sistema real d'emergències a gran escala.

La metodologia consistirà a partir de l'escenari base (5 centres i 100 grups) i anirem augmentant proporcionalment la mida del problema de 5 en 5 centres i de 100 en 100 grups tal i com diu l'enunciat, fins a arribar a un escenari massiu de 25 centres i 500 grups. S'executarà cadascun dels algorismes per a cada escenari sobre les mateixes 10 llavors aleatòries, extreient-ne la mitjana del temps d'execució i del cost heurístic. Les condicions inicials seran idèntiques per a tots dos: inicialització Avariciosa i Heurística 1. Per al

Simulated Annealing, utilitzarem la configuració òptima trobada a l'Experiment 3 ($k=4$, $\lambda=0.00002$ i 100.000 iteracions).

En aquest experiment enfrontem directament tots dos algorismes, per la qual cosa hem formulat dues hipòtesis de partida:

1. Hipòtesi sobre la qualitat de la solució:

Plantegem que el Hill Climbing serà capaç de trobar solucions de major qualitat (costos heurístics inferiors) que el Simulated Annealing als escenaris més grans.

L'espai d'estats d'aquest problema creix de manera factorial a mesura que augmentem els grups i els helicòpters. Això es deu al fet que no només importa a quin helicòpter s'assigna cada grup, sinó l'ordre exacte en què són visitats. El nombre de permutacions possibles per a G grups es troba en l'ordre de magnitud de:

$$O(G!)$$

La justificació de la nostra hipòtesi recau en el fet que el HC aplica un gradient descendent estricte, avaluant exhaustivament tots els veïns possibles a cada pas i triant el millor. Per l'altra banda, el SA explora mitjançant un camí estocàstic; si bé 100.000 iteracions eren suficients per recórrer un tauler de 100 grups i convergir cap al mínim global, en un tauler massiu de 500 grups l'espai d'estats és tan inabastable que és molt probable que el SA esgoti les seves iteracions abans de trobar una ruta tan polida com la del HC.

Hipòtesi final: S'espera que el Hill Climbing sigui capaç de trobar solucions de major qualitat, és a dir, costos heurístics inferiors, que el Simulated Annealing als escenaris més grans gràcies a la seva avaluació exhaustiva de l'entorn.

2. Hipòtesi sobre l'evolució del temps d'execució i memòria

Per predir el comportament computacional, hem d'analitzar matemàticament la complexitat temporal i espacial de la nostra funció de successors a cada pas de l'algorisme:

• Hill Climbing

L'operador del HC implementat genera tots els moviments possibles mitjançant dues accions clàssiques.

D'una banda, l'intercanvi de dos grups qualssevol de posició o helicòpter. Les combinacions possibles per triar 2 grups entre G vénen donades pel coeficient binomial, que creix de forma quadràtica:

$$Successors_{swap} = \binom{G}{2} = \frac{G \cdot (G-1)}{2} \approx O(G^2)$$

D'altra banda, el desplaçament (Move) d'un grup a un altre helicòpter, que per a G grups i H helicòpters disponibles genera

$$Successors_{move} \approx O(G \cdot H)$$

Sumant ambdós operadors, el factor de ramificació complet (b) del HC a cada pas és de la forma:

$$O(G^2 + G \cdot H)$$

Per tant, en l'escenari massiu de 500 grups, el HC ha d'instanciar, guardar a la memòria RAM i avaluar més de 125.000 nodes simultàniament només per fer un sol pas.

- **Simulated Annealing:**

A diferència del HC, l'operador del SA només genera un sol successor aleatori per iteració per decidir si l'accepta o no, independentment de com de gran sigui el tauler. Això fa que la seva complexitat espacial sigui sempre de $O(1)$ (cost constant). El temps vindrà donat per la fórmula:

$$Temps_{SA} \approx Iteracions \cdot O(Cost\ Heuristica)$$

Donat que hem limitat el SA a 100.000 iteracions, formulem la hipòtesi que el seu creixement dependrà únicament de l'increment de temps que triga la funció heurística en llegir un tauler més gran.

Hipòtesi final: S'espera que el temps d'execució del Simulated Annealing escali de manera lineal, suau i totalment assumible per a la memòria. Al contrari, el Hill Climbing patirà una explosió combinatòria de creixement quadràtic a nivell de successors, la qual cosa provocarà un col·lapse i farà que el programa no es pugui executar en els escenaris més grans per esgotament de la memòria dinàmica.

2.4.2. Dades experimentals

A continuació, presentem els resultats de sotmetre ambdós algorismes a l'augment progressiu del tauler. Hem registrat tant el temps mitjà de resolució com el cost heurístic final de les 10 llavors per a cada grandària.

Cal destacar un esdeveniment crític en la darrera prova: a l'escenari de 25 centres i 500 grups, l'algorisme de Hill Climbing no ha pogut finalitzar l'execució. Ha patit un col·lapse per esgotament de la memòria RAM de la màquina virtual (java.lang.OutOfMemoryError: Java heap space) durant la generació massiva de successors, fet que ens ha impedit recollir les seves dades per a aquest nivell.

Centres	Grups	Temps d'execució HC en milisegons (mitjana)	Temps d'execució SA en milisegons (mitjana)	Cost total HC (mitjana)	Cost total SA (mitjana)
5	100	598	351	2636,28	2544,94
10	200	9.766	664	4513,98	4467,24
15	300	74.985	1.096	6478,88	6529,97
20	400	266.250	1.587	8377,62	8558,95
25	500	Out Of Memory	2.009	-	10768,72

D'aquesta taula n'extraïem observacions clares sobre el comportament dels algorismes. D'una banda, el Simulated Annealing mostra un creixement pràcticament lineal i altament eficient: passa de resoldre el primer escenari en un terç de segon a resoldre el més gran (500 grups) en tot just 2 segons (2009 ms).

En canvi, el temps del Hill Climbing creix de manera desproporcionada:

- A 100 grups, triga mig segon.
- A 200 grups, el temps es multiplica per 16 (gairebé 10 segons).
- A 300 grups, es dispara a més d'1 minut i 14 segons.
- A 400 grups, triga més de 4 minuts i mig (266 segons) per a cada llavor.

Finalment, a 500 grups, excedeix l'espai de memòria.

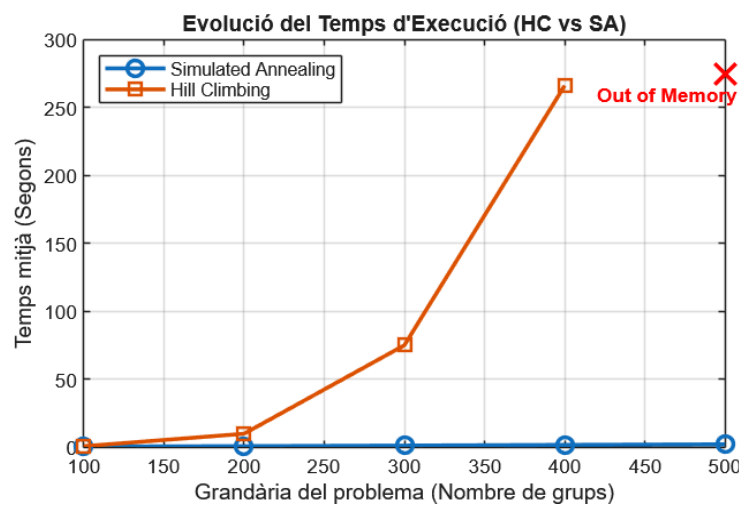


Figura 6: Evolució del temps d'execució en funció de la grandària del problema

Pel que fa al cost heurístic, podem observar un "punt de tall" o sorpasso. En els problemes petits (100 i 200 grups), el SA optimitza millor la ruta. Ara bé, a partir dels 300 grups, la cerca exhaustiva del HC comença a donar els seus fruits i aconsegueix costos heurístics inferiors al SA, superant-lo per gairebé 200 minuts a l'escenari de 400 grups.

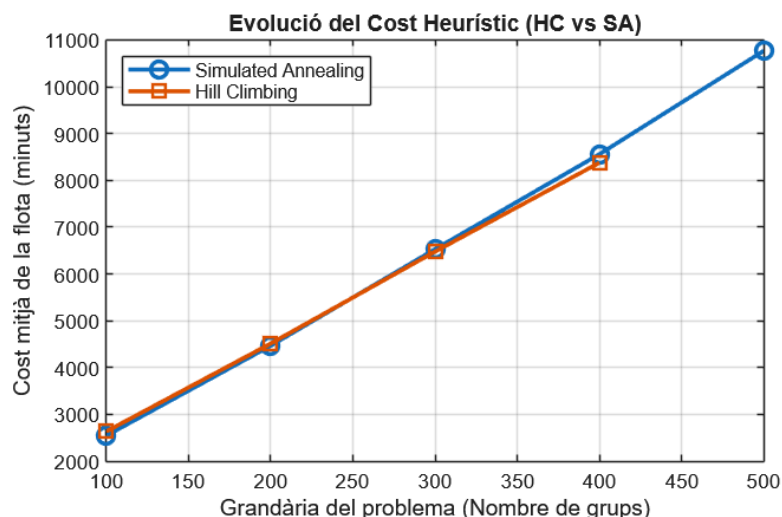


Figura 7: Evolució del cost heurístic en funció de la grandària del problema

2.4.3. Anàlisi

Tal com recull la taula de resultats, el Hill Climbing no mostra cap superioritat en els escenaris petits i mitjans (100 i 200 grups). En aquests primers nivells, on l'espai d'estats és contingut, les 100.000 iteracions del Simulated Annealing són més que suficients perquè la seva capacitat d'evitar òptims locals guanyi la partida i trobi millors assignacions.

Tot i això, a mesura que el tauler creix cap a dominis massius (300 i 400 grups), es produeix el "punt de tall" previst. Tal com preveia la primera hipòtesi, l'exploració aleatòria del SA es comença a diluir dins d'un espai d'ordre factorial $O(G!)$, i és precisament aquí on la voracitat exhaustiva del Hill Climbing dóna els seus fruits. En el cas de 400 grups, el HC aconsegueix superar la qualitat de la solució del SA, rebaixant el temps de la flota de rescat en quasi 200 minuts. Per tant, s'accepta la hipòtesi de la qualitat. El SA domina els espais petits i mitjans, però el HC, gràcies a l'exploració exhaustiva de tot el veïnatge, aconsegueix solucions més polides quan la complexitat augmenta.

Si bé la primera conclusió donava el punt de qualitat al HC en problemes grans, els resultats de rendiment el descarten com a opció viable quan l'escala augmenta. Com es pot observar en el gràfic d'evolució del temps a l'apartat anterior, el Simulated Annealing demostra una escalabilitat pràcticament perfecta i lineal. Resoldre un escenari cinc vegades més gran només li ha suposat multiplicar per sis el seu temps d'execució inicial. Tal com vam deduir, el cost computacional $O(1)$ del seu operador de generació el fa invulnerable a l'explosió de l'espai d'estats.

En contrast dramàtic, les dades de temps i memòria del Hill Climbing segueixen al mil·límetre la nostra deducció del creixement del factor de ramificació quadràtic:

$$O(G^2 + G \cdot H)$$

Les dades experimentals mostren un augment salvatge de la corba temporal i una càrrega inassumible a memòria. La prova definitiva es materialitza en el nivell màxim (500 grups), on la necessitat d'instanciar i mantenir centenars de milers de successors simultanis per prendre una sola decisió rebenta l'espai d'emmagatzematge dinàmic de la màquina virtual de Java (Heap Space Error). Per tant, també es confirma la hipòtesi d'explosió combinatoria del HC enfront de l'eficiència constant i estable del SA.

D'aquesta manera hem demostrat que l'escala del problema té un impacte crític en l'elecció de l'algorisme. Tot i que el Hill Climbing és capaç de prendre una millor qualitat en escenaris grans explorant cada veïnatge al màxim, pateix un col·lapse espacial i temporal que el fa insostenible. Al contrari, si tenim en compte aquest problema, l'arquitectura del Simulated Annealing es proclama com la guanyadora absoluta d'aquesta comparativa d'escalabilitat: la seva capacitat de gestionar un problema de dimensions massives (500 grups) de forma estable en només dos segons de còmput, sense cap risc de col·lapse per manca de memòria, el converteix en l'enfocament idoni per a escenaris d'alta exigència computacional. A més a més, proporciona solucions millors en escenaris petits i mitjans, i no és fins a l'escenari amb 400 grups que comença a decaure, però encara no és una pèrdua de "bondat" gaire notable. El cost de 8377,62 minuts del HC contra els 8558,95 del SA suposa

una diferència de gairebé 200 minuts que, tot i ser unes 3 hores de diferència, hem de tenir en compte la magnitud, i és que tant 8377 minuts com 8559 són gairebé 6 dies, una diferència de 3 hores no és gaire significativa.

2.5. Experiment 5

2.5.1. Plantejament

Ens interessa també estudiar com evoluciona el temps d'execució per a valors creixents dels paràmetres del problema de forma separada. Estudiarem individualment com la variació del *nombre de grups a rescatar* i dels *centres de rescat disponibles* té un efecte sobre el temps d'execució.

La metodologia que seguirem serà mesurar el temps d'execució de la funció de cerca a nivell de codi (utilitzant la funció *System.nanoTime()*). Després, observarem el seu efecte en augmentar un d'aquests dos paràmetres (de 50 en 50 en el primer cas, o de 5 en 5 en el segon). Quan detectem una tendència clara en els temps d'execució, deixarem d'augmentar la mostra.

Tal i com dicta l'enunciat, cada mesura del temps d'execució es farà mesurant 10 llavors diferents amb el mateix valor d'un paràmetre seleccionat, i posteriorment es farà la mitja d'aquests valors.

En quant a la hipòtesi de quin resultat pot ser esperat per aquest experiment, donat que es plantegen dos escenaris diferents: augment de grups a rescatar i augment de centres de rescat, tindrem dues hipòtesis diferents sobre com evolucionarà el temps d'execució. Una per cada paràmetre:

1. Hipòtesi de l'evolució del temps d'execució respecte al nombre de grups

Per analitzar el temps d'execució en funció del nombre de grups, hem d'analitzar l'estructura de l'algorisme de cerca escollit, que en aquest cas és l'algorisme de Hill Climbing.

Recordem de la teoria que Hill Climbing té la següent forma:

```
Algoritmo: Hill Climbing  
Actual ← Estado_inicial  
fin ← falso  
mientras no fin hacer  
| Hijos ← generar_sucesores(Actual)  
| Hijos ← ordenar_y_eliminar_peores(Hijos, Actual)  
| si no vacio?(Hijos) entonces  
| | Actual ← Escoger_mejor(Hijos)  
| sino  
| | fin ← cierto
```

Figura 8: Referència del material de l'assignatura de l'algorisme de Hill Climbing

A partir de l'estructura d'aquest algorisme, podem deduir els paràmetres dels quals depèn el temps d'execució. Observem que el cost del bucle és:

$$N. \text{ de iteracions } \times N. \text{ de successors per iteració}$$

I observem també que el nombre de successors és dependent de la quantitat de combinacions possibles generades pels nostres operadors, per tant es podrà reescriure en funció dels paràmetres del problema.

Els dos operadors que utilitzem són moureGrup (reassignar un grup d'un helicòpter a un altre) i intercanviarGrups (agafar un grup A i un grup B i intercanviar les seves posicions a diferents helicòpters).

Veiem llavors que l'espai de combinacions generat per aquests dos operadors és, en el primer cas, $N. \text{ grups} \times N. \text{ helicopters}$ i en el segon, una combinatòria dels grups agafats de dos en dos: $\frac{N. \text{ grups} \times (N. \text{ grups} - 1)}{2} \approx N. \text{ grups}^2$

Per tant, el nombre de successors està relacionat al nombre de grups de la següent forma:

$$N. \text{ de successors} \approx N. \text{ grups} \times N. \text{ helicopters} + N. \text{ grups}^2 \approx N. \text{ grups}^2$$

Llavors podem expressar el temps en funció del nombre de grups de la següent forma:

$$Temps d'execucio \approx N. \text{ de iteracions } \times N. \text{ grups}^2$$

Fem notar que la variable $N. \text{ de iteracions}$ també pot estar subjecte a canviar amb l'augment del nombre de grups, ja que si hi ha més grups a rescatar el problema és més difícil. Per tant, potencialment el cost podria arribar a ser $N. \text{ grups}^3$.

Tanmateix el que esperem és que el cost asimptòtic sigui igual o una mica pitjor que el quadràtic.

Hipòtesi final: El temps d'execució és quadràtic (o lleugerament pitjor que el quadràtic) respecte el nombre de grups.

2. Hipòtesi de l'evolució del temps d'execució respecte al nombre de centres.

Anteriorment, hem vist que el temps d'execució de l'algorisme Hill Climbing es pot expressar com a:

$$Temps d'execucio \approx N. \text{ de iteracions } \times N. \text{ de successors}$$

I el nombre de successors està relacionat amb el nombre de grups i el nombre d'helicòpters de la següent forma:

$$N. \text{ de successors} \approx N. \text{ grups} \times N. \text{ helicopters} + N. \text{ grups}^2$$

Donades les condicions inicials d'aquest experiment (tenim un taulell amb 1 helicòpter/centre), podem dir que, específicament per aquest experiment, que el nombre d'helicòpters i el nombre de centres és el mateix. Llavors:

$$N. \text{ de successors} \approx N. \text{ grups} \times N. \text{ de centres} + N. \text{ grups}^2 \approx N. \text{ de centres}$$

Per tant, el temps d'execució està relacionat amb el nombre de centres de la següent forma:

$$\text{Temps d'execució} \approx N. \text{ de iteracions} \times N. \text{ de centres}$$

Amb aquesta nova expressió veiem que el temps d'execució està relacionat linealment al nombre de centres.

De la mateixa manera que en l'escenari anterior, fem notar que el nombre d'iteracions també pot dependre del nombre de centres. L'addició de més centres de rescat simplifica la dificultat del problema, per tant, potencialment el nombre d'iteracions pot disminuir amb l'augment de centres.

Hipòtesi final: El temps d'execució és linealment proporcional (o lleugerament millor) al nombre de centres.

Finalment, les condicions inicials amb les que farem l'experiment són:

- Algorisme de cerca: Hill Climbing
- Tipus d'inicialització: Avariciosa
- Heurística: Basada en el criteri 1
- Tauler: 5 (+5) centres de rescat, 1 helicòpter per centre i 100 (+50) grups a rescatar

2.5.2. Dades experimentals

1. Sobre el nombre de grups

En la següent taula podem veure els temps d'execució mitjans resultants de cercar solucions per un nombre de grups dins del rang de 100 a 400 grups.

Nº de grups	100	150	200	250	300	350	400
Temps d'execució en segons (mitja)	0,3865	1,9811	6,9430	16,3080	49,4085	91,9337	160,2386
Desviació típica	0,1038	0,3476	0,6613	1,9189	5,6772	9,0872	13,7609

Aquestes dades confirmen que el temps d'execució mostra un comportament quadràtic respecte al nombre de grups.

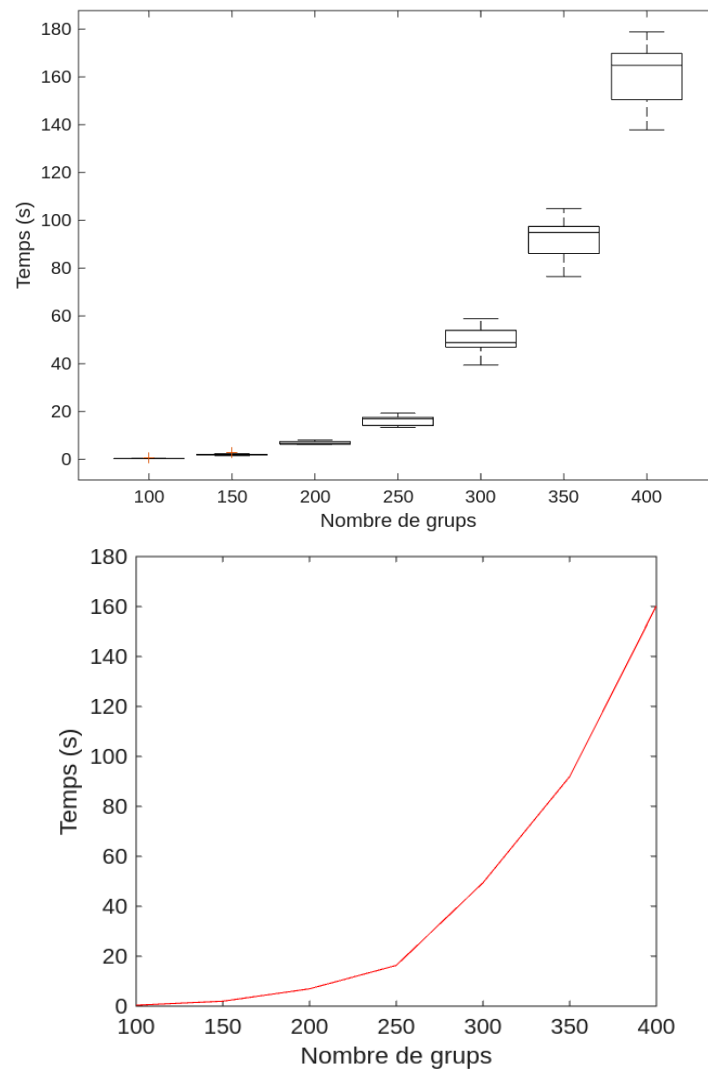


Figura 9: Evolució del temps d'execució en funció del nombre de grups

A més de comprovar-ho visualment, podem veure que el comportament és quadràtic ajustant una funció quadràtica a la mostra mesurada.

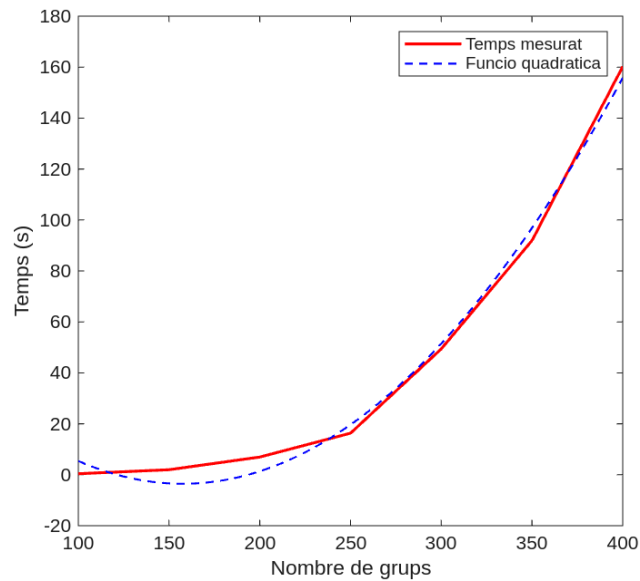


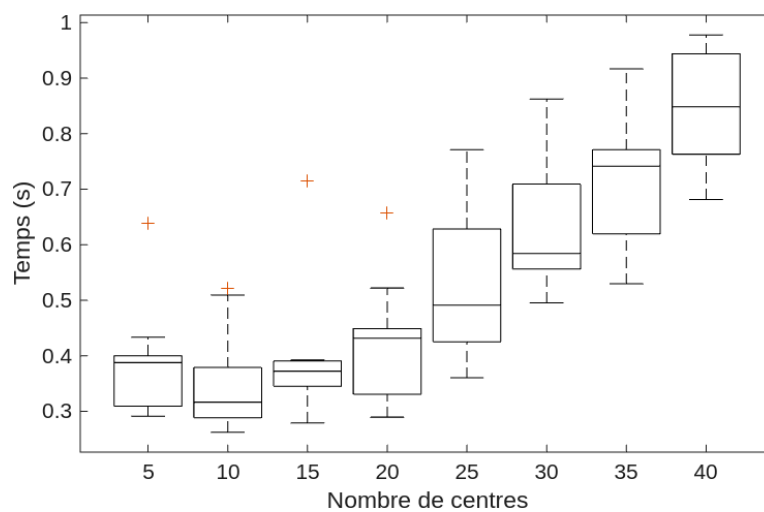
Figura 10: Evolució del temps d'execució ajustat a una funció quadràtica

2. Sobre el nombre de centres

En la següent taula podem veure els temps d'execució mitjans resultants de cercar solucions per a un nombre de centre dins del rang de 5 a 40 centres.

Nº de centres	5	10	15	20	25	30	35	40
Temps d'execució en segons (mitja)	0,3884	0,3490	0,3907	0,4182	0,5227	0,6245	0,7172	0,8455
Desviació	0,101	0,0936	0,1195	0,1143	0,128	0,1146	0,1101	0,1015

Veiem que el temps disminueix una mica inicialment, i, a partir dels 20 centres passa a augmentar linealment.



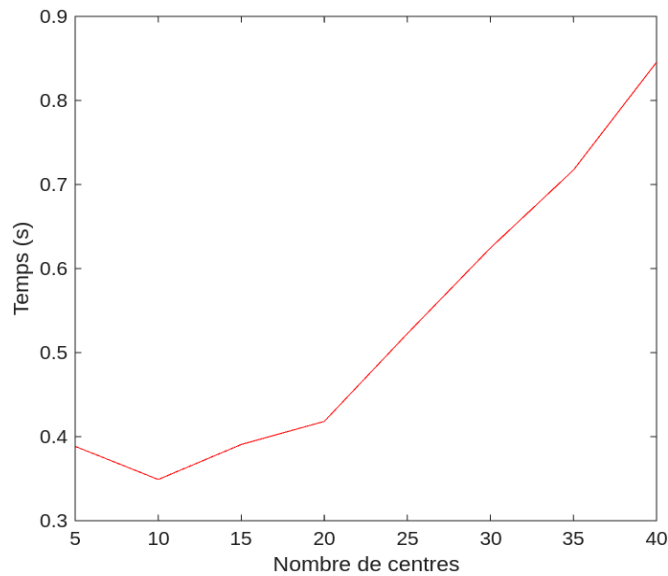


Figura 11: Evolució del temps d'execució segons el nombre de centres

També ho podem veure de forma més clara aquesta tendència representant la funció lineal gràficament.

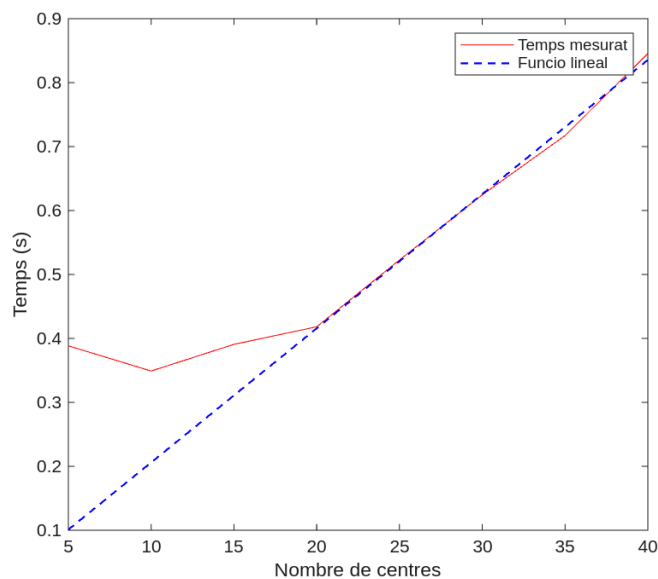


Figura 12: Temps d'execució segons el nombre de centres ajustat a una funció lineal

2.5.3. Anàlisi

1. Sobre el nombre de grups

Veiem que la hipòtesi inicial: el temps d'execució augmenta quadràticament respecte el nombre de grups, es compleix de forma bastant acurada. Potencialment, podem teoritzar que amb un augment encara major dels grups a rescatar, la dificultat amb la que es trobaria la solució es manifestaria molt més, i veuríem un creixement cúbic del temps d'execució. Tanmateix, no és observable en la mostra mesurada.

2. Sobre el nombre de centres

Observem que la hipòtesi inicial: el temps d'execució és directament proporcional al nombre de centres, només es compleix parcialment. El temps d'execució augmenta de forma lineal a partir dels 20 centres, però disminueix lleugerament al voltant dels 10 centres.

El motiu d'aquest comportament possiblement sigui el fet que al voltant dels 10 centres es troba la quantitat mínima òptima de centres per distribuir tots els ferits. Això pot causar que la solució sigui trobada molt ràpidament, ja que tenir més centres rebaixa la dificultat de la solució, sense a la vegada explorar un espai de successors massa gran en cada iteració.

Com hem dit abans, Hill Climbing depèn del nombre d'iteracions (a més del nombre de successors), per tant, aquesta reducció del nombre d'iteracions causa que es redueixi el cost total de la solució entre 0 i 20 centres.

Per una quantitat de centres major a 20, possiblement l'addició de més centres sigui redundant i, per tant, la facilitat amb la que es troba la solució (nombre d'iteracions) es manté constant. Això causa que el temps creixi linealment amb l'augment de l'espai de successors.

2.6. Experiment 6

2.6.1. Plantejament

En aquest experiment, enlloc de mesurar la quantitat de recursos com era el cas amb els paràmetres *nombre de grups* i *nombre de centres*, mesurarem la correcta configuració de recursos (nombre d'helicòpters/centre) i veurem quin impacte té en la solució trobada i el temps d'execució.

Llavors haurem de fer dos experiments: mesurar el temps de rescat i d'execució, mantenint el nombre de centres i grups constant, i mesurar el temps de rescat variant el nombre de centres i d'helicòpters per centre (de tal forma que el total d'helicòpters es mantingui constant). Utilitzarem les següents condicions inicials pels dos experiments:

- Algorisme de cerca: Hill Climbing
- Tipus d'inicialització: Avariciosa
- Heurística: Basada en el criteri 1
- 100 grups de rescat

Durant els experiments variarem el nombre d'helicòpters per centre i prendrem mesures amb deu llavors diferents per a cada valor.

Abans de començar podem formular les següents tres hipòtesis:

1. Hipòtesi sobre el temps de rescat (solució) respecte al nombre d'helicòpters

L'augment d'helicòpters per centre facilita el rescat de ferits en menys temps, per tant, és d'esperar que quan augmenti el nombre d'helicòpters el temps de rescat disminueixi.

Tanmateix, hi haurà un moment en el qual augmentar la quantitat d'helicòpters ja no beneficiï al temps de rescat, perquè hem sobrepassat la quantitat òptima d'helicòpters. Un cop arribats a aquest punt, veuríem que el temps es manté constant malgrat que afegim més helicòpters per centre.

Hipòtesi final: El temps de rescat disminuirà amb l'augment del nombre d'helicòpters, fins a un cert punt, a partir d'aquell punt es mantindrà constant.

2. Hipòtesi sobre el temps d'execució respecte al nombre d'helicòpters

En l'experiment 5 hem comentat que el temps d'execució de l'algorisme Hill Climbing, seguia la següent expressió en aquest problema:

$$\text{Temps d'execució} \approx N. \text{ de iteracions} \times N. \text{ de successors}$$

I el nombre de successors està relacionat amb el nombre de grups i el nombre d'helicòpters de la següent forma:

$$N. \text{ de successors} \approx N. \text{ grups} \times N. \text{ helicòpters} + N. \text{ grups}^2$$

Per tant, el temps d'execució respecte el nombre d'helicòpters és el següent:

$$\text{Temps d'execució} \approx N. \text{ de iteracions} \times N. \text{ de helicòpters}$$

Que es pot reescriure com a:

$$\text{Temps d'execució} \approx N. \text{ de iteracions} \times N. \text{ de centres} \times N. \text{ de helicòpters/centre}$$

I com que el nombre de centres en aquest experiment és constant:

$$\text{Temps d'execució} \approx N. \text{ de iteracions} \times N. \text{ de helicòpters/centre}$$

Finalment, amb aquesta expressió, podem formular la hipòtesi de que el temps d'execució creix linealment respecte al nombre d'helicòpters per centre.

Val la pena senyalar, però, que el nombre d'iteracions també pot disminuir quan afegim helicòpters, ja que serà més fàcil trobar la solució. Per tant, mentre la solució vagi millorant (no s'estanqui en un valor constant) por ser que observem un temps d'execució millor que el lineal. Quan la solució ja assoleixi el seu valor òptim, el temps serà directament proporcional al nombre d'helicòpters per centre.

Hipòtesi final: Lineal o millor fins a arribar al nombre d'helicòpters òptim i després lineal.

3. Hipòtesi sobre si és el mateix augmentar el nombre de centres amb un sol helicòpter

No és el mateix. Augmentar el nombre de centres amb un sol helicòpter expandeix l'àrea immediata de rescat dels helicòpters (reduïx la distància mitja al ferit més proper). En canvi, augmentar el nombre d'helicòpters per centre augmenta la capacitat d'aquell centre de rescatar els ferits que l'envolten. Un expandeix la zona d'influència i l'altre la capacitat dels centres.

L'estratègia òptima serà un punt mig de les dues estratègies.

2.6.2. Dades experimentals

1. Sobre el temps de rescat (solució)

Nº d'helicòpters/centre	1	5	10	15	20	25	30	35	40
Temps total (mitja)	2617,7	2364,2	2149,9	2094,3	2064,9	2050,3	2053,4	2051,9	2050,5

Veiem que el temps de rescat és inversament proporcional al nombre d'helicòpters per centre.

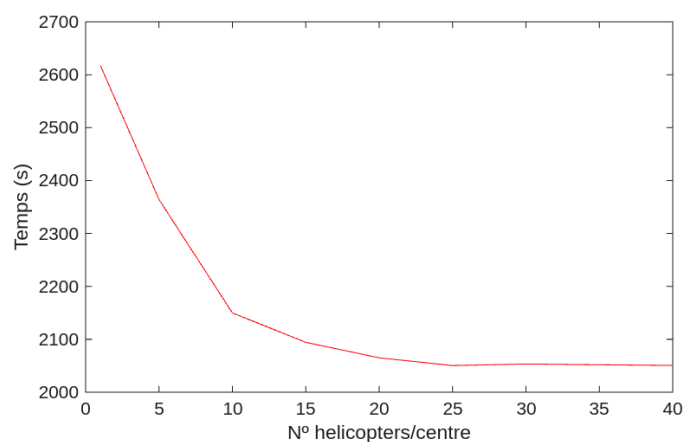


Figura 13: Evolució del temps de rescat segons el nombre d'helicòpters per centre

2. Sobre el temps d'execució

Nº d'helicòpters/centre	1	5	10	15	20	25	30	35	40
Temps d'execució en segons (mitja)	0,3881	0,6298	1,2330	1,8181	2,5282	3,2321	6,2944	8,1063	9,0017
Desviació	0,1083	0,1422	0,2015	0,2682	0,3514	0,4310	0,6258	0,4474	1,2580

Veiem que el temps d'execució és lineal fins a arribar als 25 helicòpters, després dels quals segueix augmentant, però amb un pendent major i de forma més irregular.

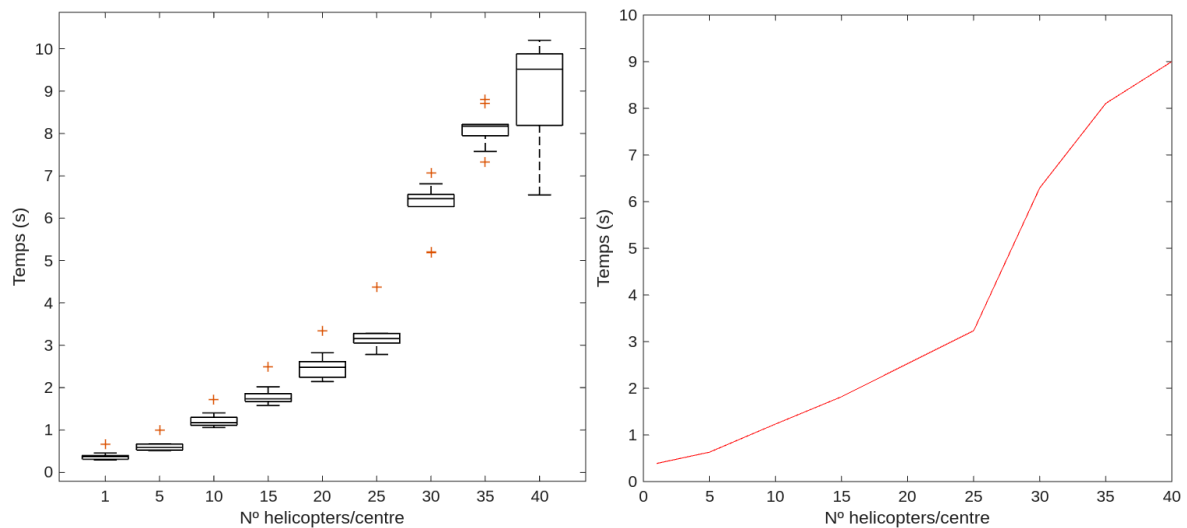


Figura 14: Evolució del temps d'execució en funció del nombre d'helicòpters per centre

3. Sobre l'estratègia de distribució d'helicòpters

Nº d'helicòpters total	5	25	50	75	100	125	150
Temps total 1(5 centres x N. heli)	2617,7	2364,2	2149,9	2094,3	2064,9	2050,3	2053,4
Temps total 2 (N. centres x 1 heli)	2617,7	2015,4	1792,5	1697,3	1652,1	1625,7	1608,7

Veiem que, respecte a tenir 5 centres amb diversos helicòpters, és molt més beneficiós per a la solució tenir més centres amb 1 helicòpter.

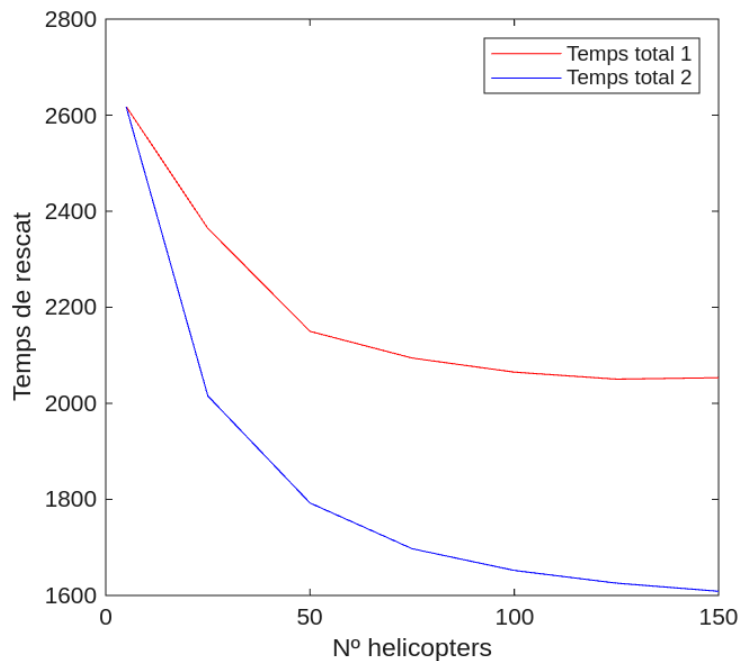


Figura 15: Evolució del temps de rescat en funció del Nº d'helicòpters per dos tipus de configuracions de centres i helicòpters.

2.6.3. Anàlisi

Després de dur a terme l'experiment, podem afirmar amb seguretat que el nombre òptim d'helicòpters per centre són 25, ja que és el punt on la solució deixa de millorar i el punt on el temps d'execució es dispara.

En quant a les hipòtesis inicials i fins a quin punt s'han complert, podem veure que la primera hipòtesi es compleix de la forma esperada: el temps de rescat disminueix fins a un valor constant al voltant dels 2050 minuts i 25 helicòpters.

Tanmateix, la segona hipòtesi només es compleix parcialment. Efectivament veiem que el temps d'execució és lineal i més baix abans del nombre d'helicòpters òptim (que era el que esperàvem). Però a partir dels 25 helicòpters el comportament sembla deixar de ser lineal.

La hipòtesi 3 també es compleix, els temps de rescat quan s'utilitzen molts centres d'1 helicòpter és diferent als temps de rescat quan distribuïm diversos helicòpters a 1 mateix centre. En el cas mesurat, s'observa que és millor tenir 1 helicòpter per centre i augmentar els centres que tenir 5 centres amb molts helicòpters per centre.

Tanmateix una quantitat de centres ideal podria arribar a ser millor que la configuració d'1 helicòpter per centre.

2.7. Experiment 7

2.7.1. Plantejament

En aquest experiment, volem estudiar el comportament del problema quan definim la funció heurística en funció del criteri 2 de l'enunciat. L'objectiu de la cerca esdevé minimitzar tant el temps de rescat total com temps de rescatar els grups de prioritat 1.

Per definir aquest heurístic, hem creat en el codi la funció *heurístic2()* que calcula el temps total que s'ha trigat a rescatar tots els ferits. Aquesta funció, sumada amb la funció *heurístic()*, dona un nou heurístic que minimitza el cost d'acord amb el criteri 2.

Experimentarem amb diferents combinacions ponderades d'aquests dos valors (H1 i H2) per a diferents algorismes de cerca (Hill Climbing i Simulated Annealing), i estudiarem com evoluciona la suma del temps de rescat i d'execució. Executarem Simulated Annealing amb els paràmetres $k=4$ i $\lambda=0.00002$.

1. Hipòtesi sobre el temps de rescat

El temps de rescat augmentarà a mesura que augmentem el pes donat a H2, ja que la minimització del temps de rescat (H1) tindrà menys pes a l'algorisme quant més augmentem el pes de H2.

Tanmateix, aquest augment del temps de rescat es detindrà quan deixi de ser possible minimitzar el temps de rescat dels ferits. Arribats a aquest punt, augmentar el pes de H2 no tindrà efecte sobre el resultat i la suma del temps de rescat deixarà d'augmentar.

Hipòtesi final: El temps de rescat augmentarà amb el pes de H2 fins que arribem a la solució que minimitza el temps de rescat dels ferits.

2. Hipòtesi sobre el temps d'execució

Al igual que amb el temps de rescat, el temps d'execució augmentarà amb la ponderació, ja que donar més temps a H2 fa que es compliqui el problema i que porti més temps arribar a una solució.

De la mateixa manera, quan es minimitzi al màxim el temps de rescat dels ferits, el temps d'execució deixarà d'augmentar, perquè la solució a trobar deixarà de créixer en dificultat.

2.7.2. Dades experimentals

1. Temps de rescat en funció de l'heurístic

Càlcul de l'heurístic	Temps de rescat (HC)	Temps de rescat (SA)
H1 + H2	2636,53	2761,64
H1 + 2*H2	2651,52	2833,85
H1 + 4*H2	2663,90	2881,66
H1 + 8*H2	2666,82	2990,62
H1 + 16*H2	2707,97	2990,41
H1 + 32*H2	2729,28	3020,84
H1 + 64*H2	2769,25	3002,90
H1 + 128*H2	2757,64	3008,24

Com podem veure, el temps de rescat augmenta pels dos algorismes ja que l'heurístic ja no busca minimitzar només el temps de rescat. A més, podem observar que Hill Climbing minimitza millor el temps total de rescat que Simulated Annealing.

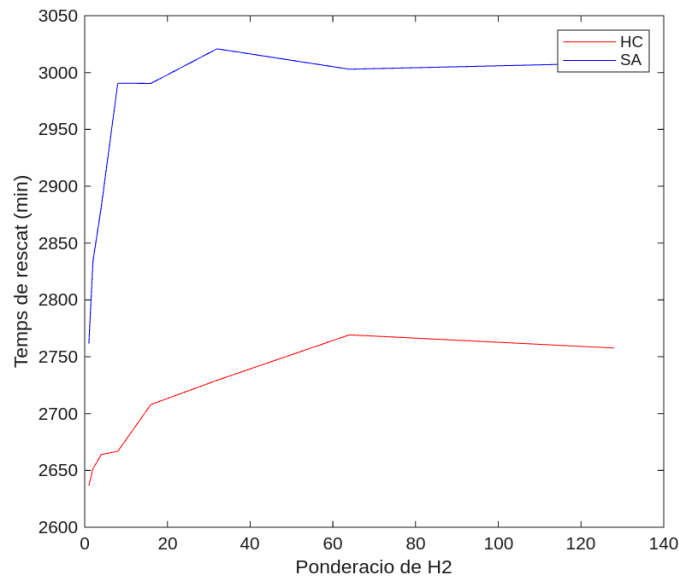


Figura 16: Evolució del temps de rescat pels algorismes de Hill Climbing i Simulated Annealing respecte a la ponderació de H2

A la ponderació 64*H2 el temps de rescat s'estabilitza, indicant que és el punt on es minimitza al màxim el temps de rescat dels ferits (l'heurístic ja no és millorable).

2. Temps d'execució en funció del càlcul de l'heurístic

	Temps d'execució (HC)	Temps d'execució (SA)
H1 + H2	0,5568	0,05487
H1 + 2*H2	0,6133	0,05536
H1 + 4*H2	0,7135	0,05541
H1 + 8*H2	0,7913	0,05541
H1 +16*H2	0,8589	0,05807
H1 +32*H2	1,0357	0,05592
H1 + 64*H2	1,1140	0,05737
H1 +128*H2	1,0742	0,05674

El temps d'execució augmenta fins a la ponderació 64*H2, ja que és el punt on es troba la solució que minimitza H2 al màxim. A partir d'aquell punt la solució trobada serà sempre la mateixa encara que augmenta el pes de H2.

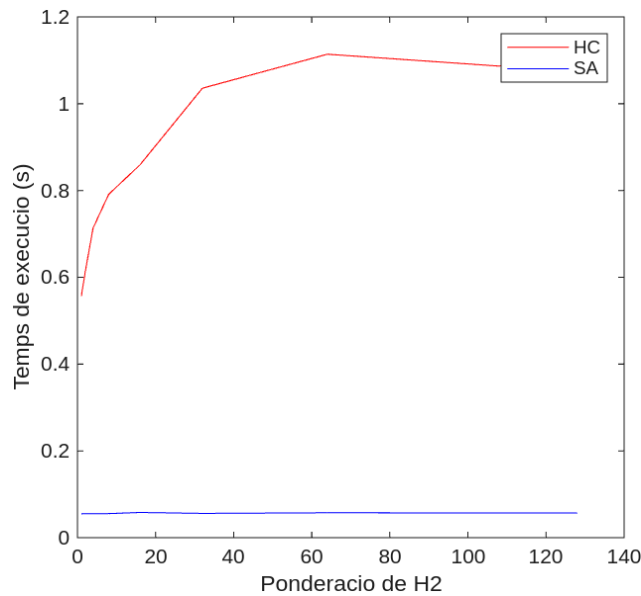


Figura 17: Evolució del temps d'execució dels algorismes de Hill Climbing i Simulated Annealing respecte a la ponderació de H2

El temps d'execució per l'algorisme de Simulated Annealing s'aprecia millor en un gràfic individual.

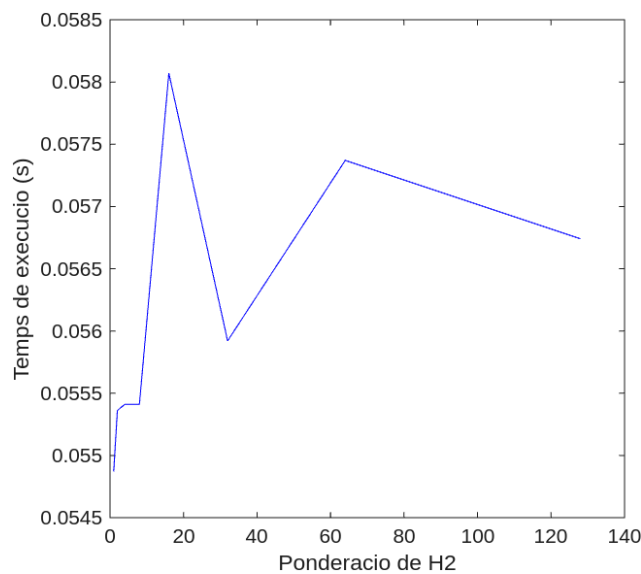


Figura 18: Evolució del temps d'execució de Simulated Annealing respecte a la ponderació de H2

Com podem veure, el temps d'execució s'estabilitza a la ponderació $64 \cdot H2$. El comportament del gràfic és lleugerament irregular perquè amb temps d'execució tan petits els resultats poden ser molt fàcilment afectats pels processos de la màquina executant l'experiment. Tot i així, es pot apreciar la tendència general d'augment i després estabilització.

2.7.3. Anàlisi

La hipòtesi sobre el temps de rescat es compleix. Podem veure que, pels dos algorismes de cerca, el temps de rescat augmenta fins a arribar a la ponderació $H1 + 64 \cdot H2$, a partir del qual el valor esdevé constant.

Aquest resultat ens indica que el primer punt en el qual es minimitza el temps de rescat dels grups de ferits és la ponderació $H1 + 64 \cdot H2$, i que no té sentit seguir donant més pes a $H2$.

També podem veure que la hipòtesi 2 es compleix: el temps d'execució, pels dos algorismes augmenten fins a arribar al valor de $64 \cdot H2$, per després regularitzar-se en un valor (Hill Climbing al voltant de 1,1s i Simulated Annealing al voltant de 0.057 segons).

3. Conclusió final

Un cop finalitzats els experiments i analitzats els resultats obtinguts en la resolució del problema de gestió d'emergències, podem extreure les següents conclusions:

Eficiència de la representació de l'estat i operadors

El fet de declarar les variables de la classe ProbBoard com a estàtiques ha estat un encert des del punt de vista de l'ús eficient de memòria al programa, perquè hem evitat que durant les iteracions de la cerca la memòria sigui ocupada per còpies de variables. D'aquesta manera la memòria del programa s'ha destinat a l'array assignacioHelicopters, que s'havia d'actualitzar dinàmicament.

En quant a la selecció d'operadors, amb els dos operadors que hem utilitzat, hem pogut observar que la millor configuració d'operadors no és cap operador de forma individual sinó la combinació dels dos. Mentre que l'operador moureGrup ajuda a reorganitzar ràpidament un grup, l'operador intercambiarGrups és capaç d'explorar zones de l'espai d'estats més allunyades i està menys limitat que moureGrup. L'operador intercambiarGrups és el que contribueix més a millorar la solució però moureGrup és més eficient.

Comparació entre Hill Climbing i Simulated Annealing

En quant a retorn de la solució òptima al problema, en l'experiment 4 veiem que els dos algorismes donen resultats molt similars. Simulated Annealing dona resultats millors amb inicialitzacions llunyanes a la solució, demostrant que quan la solució inicial no és molt bona, funciona molt millor que Hill Climbing. Tanmateix, quan utilitzem una inicialització avariciosa, els resultats dels dos algorismes són molt similars.

Hill Climbing acostuma a ser més acurat per a tamanys petits del problema, on tenir una bona solució avariciosa inicial li és suficient per trobar un bon òptim. En canvi, quan la mida del problema comença a ser més gran, Simulated Annealing dona una solució lleugerament millor.

D'altra banda, en quant a temps d'execució, Simulated Annealing és superior a Hill Climbing per diverses ordres de magnitud. Això segurament sigui una conseqüència del factor de ramificació de Hill Climbing (ha d'explorar tots els seus veïns). Simulated Annealing en canvi, només ha d'explorar un veí cada cop.

Distribució logística

En els experiments 5 i 6, hem vist que freqüentment, a l'hora de disminuir el temps de rescat i millorar la solució, en general és més beneficiós que augmentar el nombre de centres de rescat que el d'helicòpters per centre. És a dir, disminuir la distància mitja que ha de recórrer l'helicòpter per fer el rescat és més important que augmentar la capacitat dels centres, a l'hora de minimitzar el temps.

Una altra conclusió que podem extreure d'aquests experiments és que els 25 helicòpters per centre són el límit de recursos destinats al rescat necessaris. Més helicòpters serien redundants ja.

Comparació entre heurístics

Finalment, respecte a les diferents configuracions d'heurístics possibles, hem conclòs que el millor heurístic seguint el criteri 1 de l'enunciat és la suma de tots els rescats (*Heurístic 1*) i el millor heurístic seguint el criteri 2 és $Heurístic\ 1 + 64 * Heurístic\ 2$, on *Heurístic 2* és la suma de tots els rescats de grups de prioritat 1.

4. Treball d'innovació

Pel treball d'innovació hem decidit fer recerca sobre l'editor de codi Windsurf i com s'integra la generació de codi amb un llm i la creació d'arxius de codi de forma local.

De moment no hem tingut oportunitat d'avançar en el treball.